

AnomalyLLM: Few-shot Anomaly Edge Detection for Dynamic Graphs using Large Language Models

AnomalyLLM (Liu et al., 2024) presents a novel framework that detects anomalous edges in **dynamic graphs** using **few labeled examples**. It leverages the powerful generalization abilities of **frozen large language models (LLMs)**, aligning their text-based reasoning with graph-structured data through a reprogramming step. This approach is particularly useful in real-world settings where anomaly types evolve over time and labeled data is scarce.

Step 1: Representing Edges in a Dynamic Graph Context (With Example)

To make dynamic edges understandable to an LLM, AnomalyLLM first constructs **rich, context-aware representations** of graph edges. This is done using a **dynamic-aware encoder**, which captures both local structure and temporal evolution around each edge.

Example:

Consider an edge $e = (\text{Alice}, \text{Bob})$ in a network of messaging activity.

To represent this edge, the encoder builds a **structural-temporal subgraph** around Alice and Bob by:

- Collecting their top- K neighbors at each time step
- Creating subgraphs at multiple past timestamps (say, from $t-3$ to t)

This sequence of subgraphs is processed with a hybrid **GNN + Transformer** to yield a dynamic embedding r , e.g.:

$r = [0.18, -0.03, \dots, 0.41]$

Step 2: Bridging Graph Embeddings with LLMs via Reprogramming (With Example)

Since LLMs operate on text tokens—not vectors like r —AnomalyLLM uses **modality alignment** through reprogramming. It transforms the graph embedding r into a token-like space using **text prototypes**, so the LLM can process it meaningfully.

Example:

LLM is prompted to list terms relevant to dynamic graphs (e.g., “attack”, “fraud”, “benign”). These are embedded and clustered into **V text prototypes**, e.g.,

- P_1 : anomalous behavior
- P_2 : normal communication

- P_3 : unexpected time
- ...

Then, a multi-head attention mechanism uses these as keys/values and r as the query to compute:

$m = \text{Attention}(r, \{P_1, \dots, P_8\})$

This m is added to r , giving a **reprogrammed embedding** h that can be passed to the LLM.

Step 3: Few-Shot Anomaly Detection via In-Context Learning (With Example)

Now that h is in a format the LLM understands, AnomalyLLM uses **in-context learning (ICL)** to detect whether the edge is anomalous. It constructs a **prompt** with examples and feeds it to the frozen LLM.

Example Prompt:

You are a few-shot anomaly detector.

Task: Identify if a given edge represents a suspicious peer-to-peer message.

Examples:

Edge A: <embedding> → Label: Normal

Edge B: <embedding> → Label: Anomaly

Edge C: <embedding> → Label: Normal

Question:

Edge D: <embedding> → Label: ?

The LLM processes the prompt and returns a hidden vector for Edge D. This is fed into a lightweight **MLP classifier** to predict the anomaly score.

Why This Architecture Matters

Each module is carefully designed to connect with the next:

1. The **dynamic encoder** turns a raw edge into a time-aware vector r
2. The **reprogramming module** transforms r into a token-friendly vector h

3. The **ICL + MLP** block uses **h** in a natural-language prompt to predict anomalies

Importantly, the **LLM is never fine-tuned**—only the graph encoder and MLP are lightly trained. This makes the system **efficient and generalizable**.

Results and Performance Highlights

- Evaluated on UCI Messages, BlogCatalog (synthetic anomalies), and T-Finance, T-Social (real-world anomalies)
 - Outperforms state-of-the-art baselines (TGN, SAD, TADDY) in **1-shot**, **5-shot**, and **10-shot** settings
 - Maintains high AUC (>0.82) even with just **one labeled example per anomaly type**
-

Why It Matters for the Survey

AnomalyLLM is one of the first to demonstrate how **LLMs can be integrated into dynamic graph anomaly detection** using few-shot learning. Its reprogramming strategy bridges modality gaps, while ICL unlocks generalization without retraining. This makes it a standout example for LLM + Dynamic GNN integration in practical, evolving domains like fraud, intrusion, or irregular activity detection.